

Turing M/C:

The Turing m/c can be thought of as a FSA connected to a R/W head. It has one tape which is divided into a no. of cells. Each cell can store only one symbol. The input to and output from the Finite State Automata are effected by the R/W head which can examine one cell at a time. The Block diagram of the basic model for Turing m/c is given in figure below.

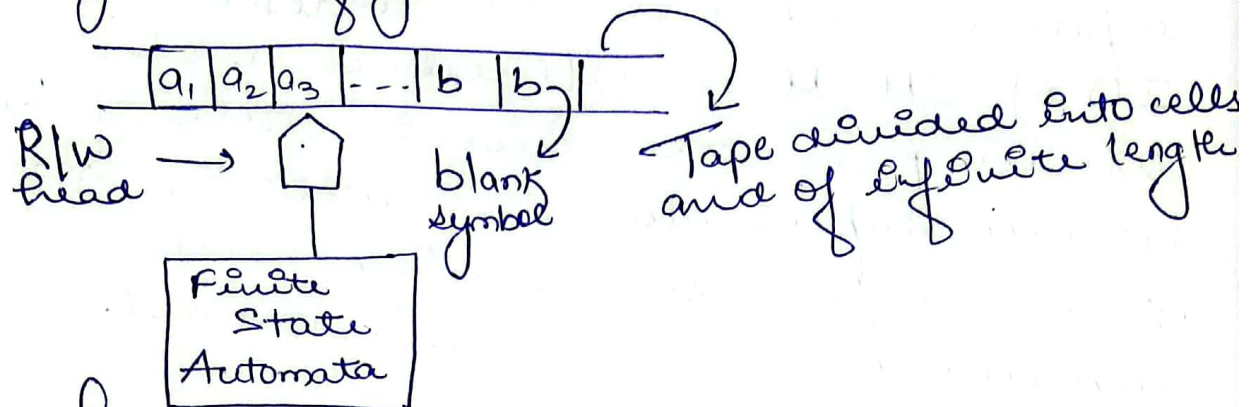


fig: Turing m/c Model.

● Definition:

A TM M has a 7 tuple $(Q, \Sigma, \Gamma, \delta, q_0, b, F)$, where

- (1) Q is a finite, non-empty set of states.
- (2) Σ is a non-empty set of input symbols and is a subset of Γ and $b \notin \Sigma$.
- (3) Γ is a finite non-empty set of Tape symbols.
- (4) $b \in \Gamma$ is the blank.
- (5) δ is the transition function, which maps.
 $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- (6) $q_0 \in Q$ is the initial state.
- (7) $F \subseteq Q$ is the set of final states.

Q3

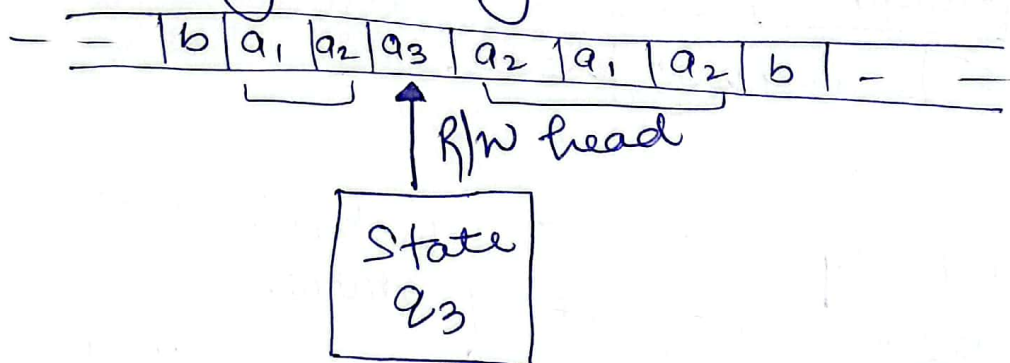
Representation of Turing Machine:

Generally we have three technique to represent the TM :-

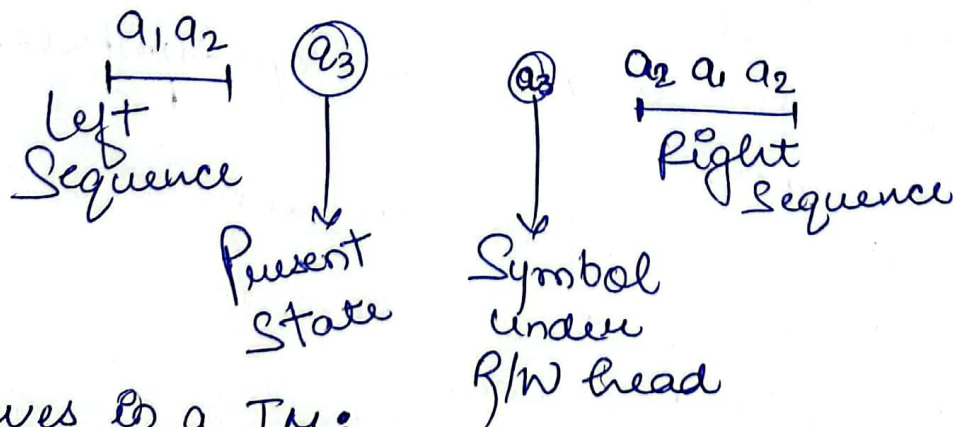
- (1) Representation by Instantaneous description:
- (2) Representation by Transition Table.
- (3) Representation by Transition Diagram.

● Representation by ID:

An ID of a TM is defined in terms of the entire input string and the current state. ~~An~~ An ID of TM M is defined as a string $\alpha\beta\gamma$ where β is the present state of TM & the entire input string is split as $\alpha\gamma$ in which α is the processed string, while γ is the remaining string to be scanned.



$$\alpha = a_1 a_2, \beta = q_3, \gamma = a_2 a_1 a_2.$$



● Moves in a TM:

$$\delta(q_1, x_3) = (q_2, y, R)$$

am

● Representation by Transition Table:

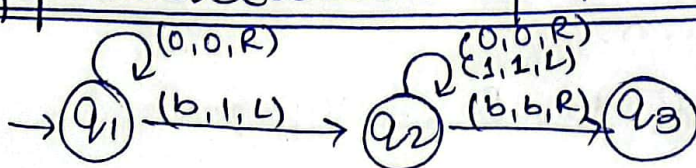
If $\delta(q, a) = (r, \alpha, \beta)$, we write $\alpha\beta r$ under a -column and q -row. So if we get $\alpha\beta r$ in the table, it means that α is written in the current cell, β gives the movement of the head (L or R) and r denotes the new state into which the turning m/c enters.

Present State	Type Symbols		
	b	0	1
$\rightarrow q_1$	1L q_2	0R q_1	—
q_2	bR q_3	0L q_2	1L q_2
q_3	—	bR q_4	bR q_5
q_4	0R q_5	0R q_4	1R q_4
* q_5	0L q_2	—	1R q_4

Ques Draw the Computation Sequence of Input string 00.

Ans $q_1 00b \vdash 0q_1 0b \vdash 00q_1 b \vdash 0q_2 01 \vdash q_2 001$
 $q_2 b001 \vdash bq_3 001 \vdash bbq_4 01 \vdash bb0q_4 1 \vdash bb01q_4 b$
 $\vdash bb010q_5 b \vdash bb01q_2 00 \vdash bb0q_2 100 \vdash$
 $bbq_2 0100 \vdash bq_2 b0100 \vdash bbq_3 0100 \vdash bbbq_4 100$
 $\vdash bbb1q_4 00 \vdash bbb10q_4 0 \vdash bbb100q_4 b \vdash$
 $bbb1000q_5 b \vdash bbb100q_2 00 \vdash bbb10q_2 000 \vdash$
 $bbb1q_2 0000 \vdash bbbq_2 10000 \vdash bbb2b10000 \vdash$
 $bbbq_3 10000 \vdash bbbbq_5 0000$

● Representation by Transition Diagram:



q5

#Variants of TM:

● Multitape Turing Machines:

A Turing machine with several input tapes is said to be a multitape TM. In a multitape TM each tape is controlled by its own independent R/W (Read/Write) head. A multitape TM with k tapes is shown in fig.

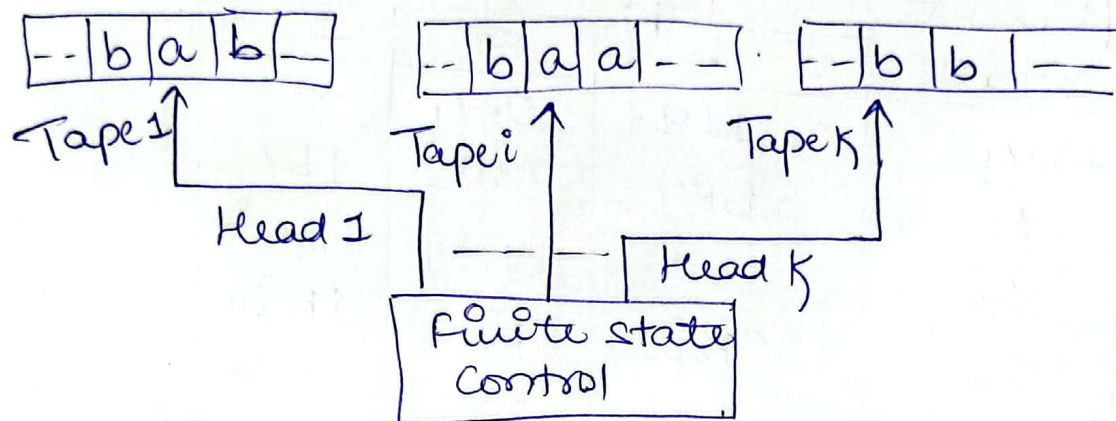


fig: A multitape TM model

Typically an n -tape TM can be defined as $M = (Q, \Sigma, \delta, \Gamma, q_0, b, F)$ where all the parameters have the same meanings as in the basic definition of a TM but the transition function δ is defined $Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times [L, R, \text{stop}]^k \rightarrow \text{or No shift}$

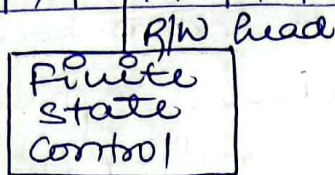
● Multidimensional Turing Machines:

A TM is said to be a multidimensional TM its input tape can be viewed as extending infinitely more than one dimension. The formal definition of a two-dimensional TM involves a transition functions of the form.

$$Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, U, D, \text{stop}\}$$

96

-	-	-	b	a ₁	a ₂	a ₃	a ₂	a ₁	a ₂	b	
-	-	-	b	b ₂	b ₃	b ₂	b ₁	b ₂	b ₂	b	
-	-	-	-	-	-	-	-	-	-	-	
-	-	-	b	k ₂	k ₁	k ₂	k ₃	k ₂	k ₁	b	



Two-dimensional input tape

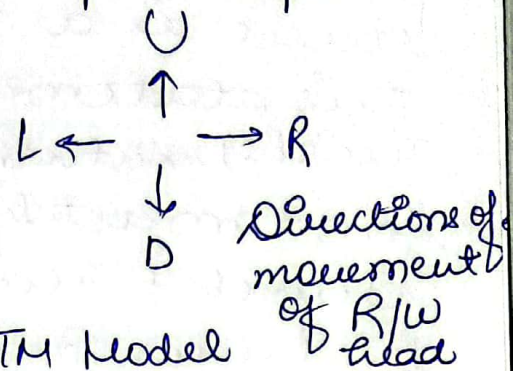


fig: A multidimensional TM Model

● Multihread Turing Machine:

A multihread TM $M = (Q, \Sigma, \delta, q_0, Z_0, F)$ can be viewed as a TM with a single tape and a single finite state control (also called unit) but the several independent R/W heads. The transition function of a multihread TM can be defined as

$Q \times \Sigma^n \rightarrow Q \times \Sigma^n \times \{L, R, \text{stop}\}^n \rightarrow \text{no. of R/W heads.}$

A move of the multihread TM depends upon the state and the symbol scanned by each head. In one move, the R/W head may take move independently L, R, or may remain stationary. A five head TM is shown in fig.

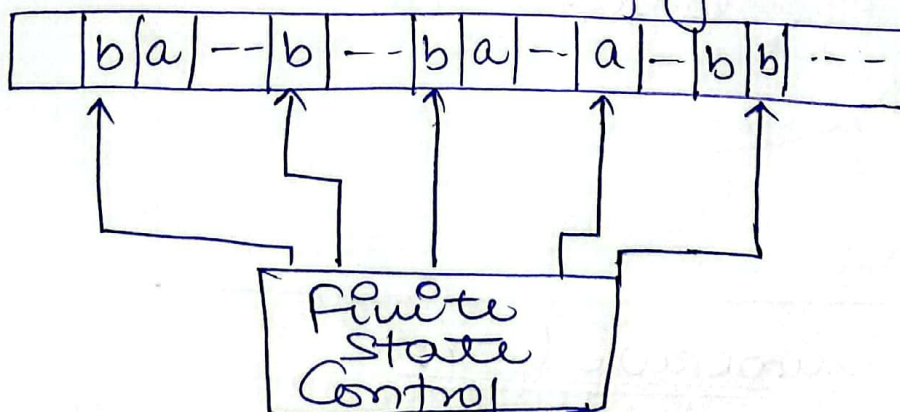


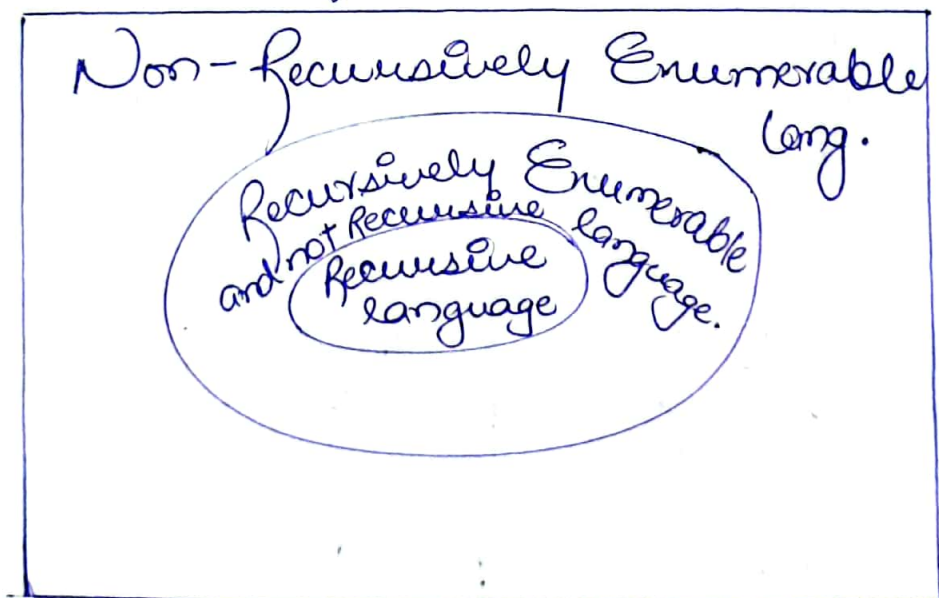
fig: A multihread TM model (five heads)

(97)

Church's Thesis:

A/c to Alonzo Church "There is an effective procedure to solve a problem iff and only iff there is a TM which solves that problem". This statement is called a thesis (Church's Thesis) rather than a theorem because it is fundamentally unprovable. Unprovable not, Church's thesis is generally accepted to be true. In fact many computer scientists define an effective procedure as something for which a TM exists. A/c to Church's Thesis, a TM can be treated as the most general computing system. Therefore, finding an algorithm for a Yes/No problem is equivalent to constructing a TM which can execute the algo.

Recursive and Recursively Enumerable lang:



● Recursively Enumerable lang:

A language is recursively enumerable if some TM accept it. Let L be a recursively enumerable language and M the TM that accept it. for string w :

(99)

if $w \in L$, then M halts in a final state.

if $w \notin L$, then M halts in a non-final state loops forever.

● Recursive language:

A lang. is recursive if some TM accepts it and halts on any input string. A language is recursive if there is a membership algorithm for it.

Let L be a recursive language and M the TM that accept it. For string w :

if $w \in L$, then M halts in a final state.

if $w \notin L$, then M halts in a non-final state.

▲ From these two languages following observations are made:

(1) Every recursive language is recursively enumerable language.

(2) Not all recursively enumerable language are recursive that means every TM which accepts the enumerable language must have some words for which it loops forever.

Universal Turing Machine (UTM):

As the name suggests universal Turing m/c is a kind of TM which is capable of doing anything that any other TM can do that means Universal TM should have capability of imitating any TM T for.

A TM is said to be universal TM if it is capable to accept all strings:

When following info is given:

99

- (1) The description of T in terms of its operation or program area of tape.
- (2) The initial configuration of T , which include starting state and the symbol scanned (state area of tape).
- (3) The input data to be given to TM, T (i.e., data area of the tape).

We introduce the notion of a universal TM (UTM) which along with the input on the tape takes in the description of a m/c M . The UTM can go on then to simulate M on the rest of the contents of the input tape. A universal TM can thus simulate any other m/c.

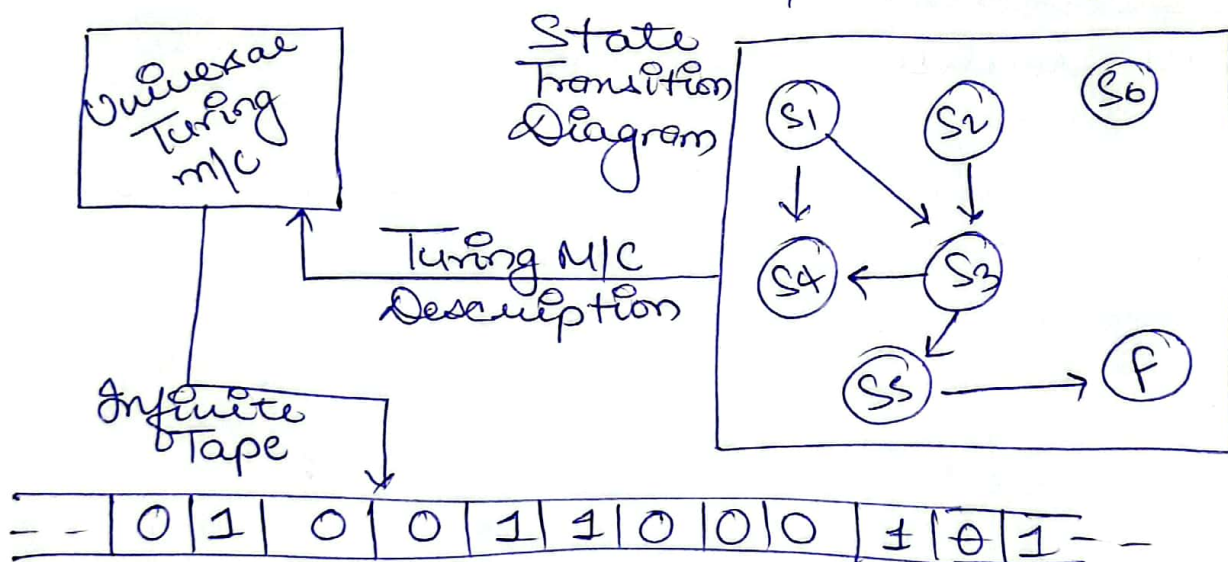
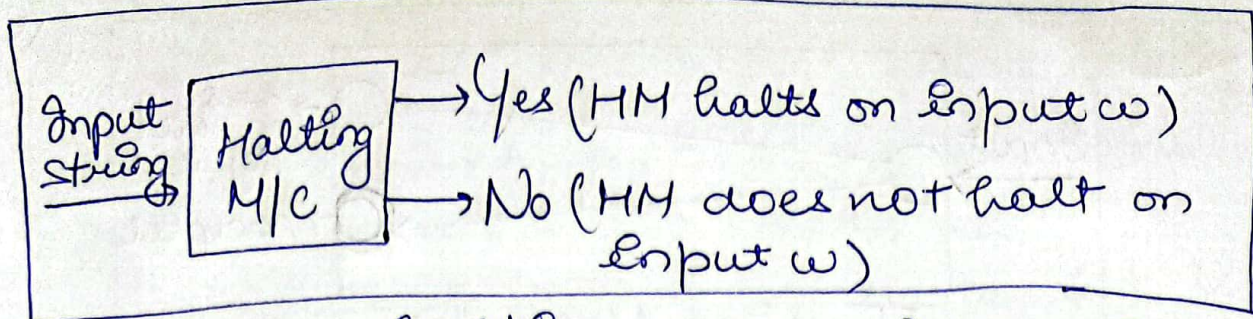


fig: UTM

Halting Problem of a TM:

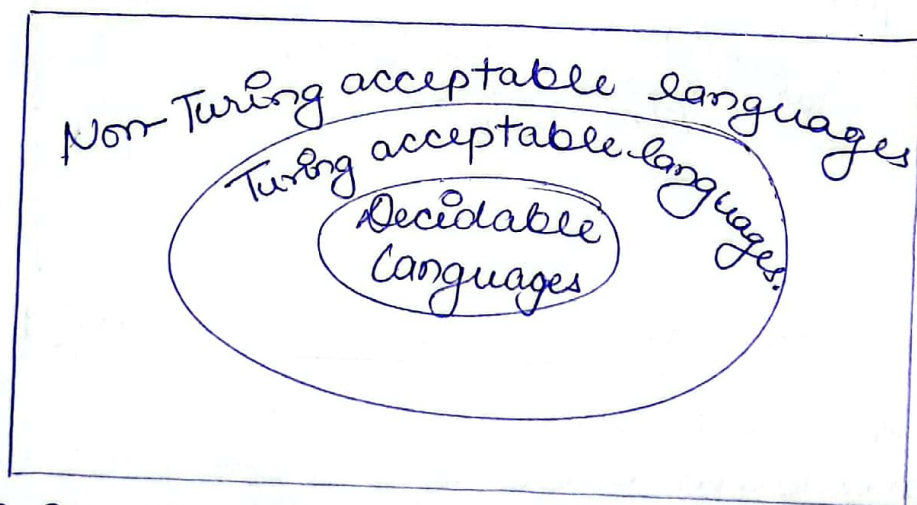
We know that input to a TM is a string. TM themselves can be written as a string and these strings can be used as an input to other m/c. While reading inputs a TM may halt or sometimes may enter in a infinite loop. In that case m/c is unable to decide what to do for a particular input.



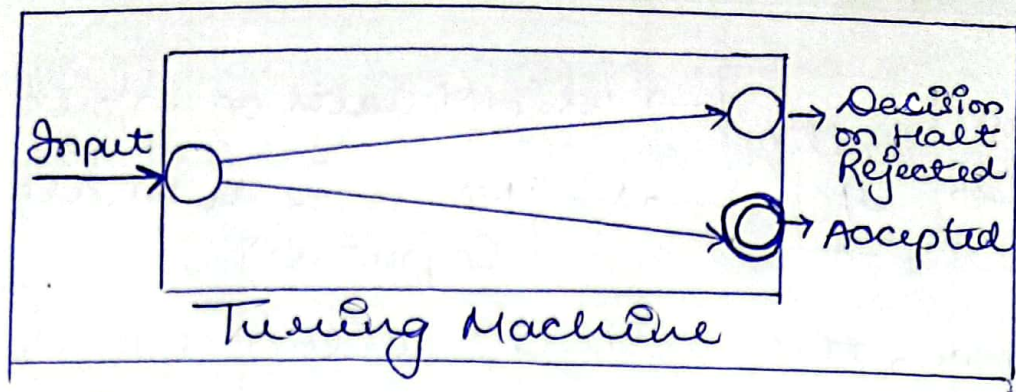
Hence, the halting problem is undecidable.

#Decidable and Undecidable Problem:

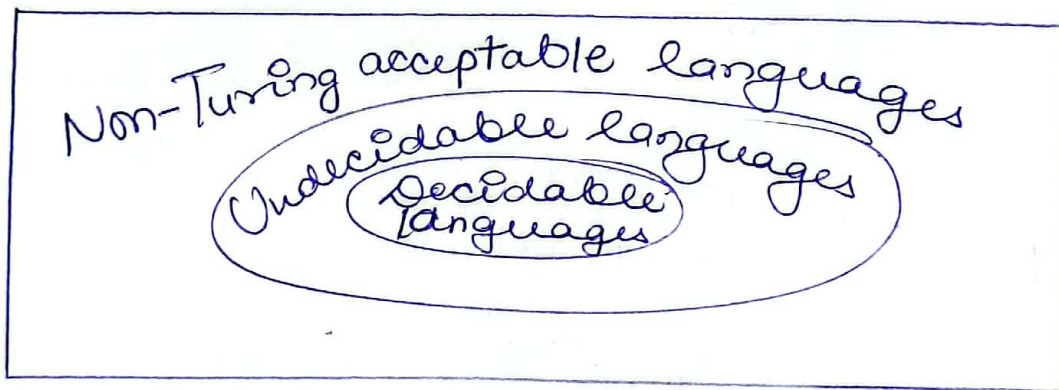
If an algorithm exist for solving a problem then this is called decidable problem. whereas when there exist no algo. to solve a problem the problem is called undecidable. A lang. is called decidable or recursive if there is a TM which accepts and halts on every input string w . Every decidable language is Turing-Acceptable



A decision problem $\langle P \rangle$ is decidable if the lang. $\langle \{ \text{all yes instances to } \langle P \rangle \} \rangle$ is decidable. For a decidable language, for each input string, the TM halts either at the accept or the reject state as depicted in the following diagram:



for an undecidable lang, there is no TM which accepts the language and makes a decision forever. Input string w . A decision problem P is called "undecidable" if the language L of all yes instances to P is not decidable. Undecidable languages are not recursive languages, but sometimes, they may be recursively enumerable languages.



#PCP (Post Correspondence problem):

The PCP was first introduced by Emil Post in 1946. The problem over an alphabet set Σ belongs to a class of Yes/No problems. The PCP consists of two strings that are of equal length over the input Σ . The two lists are $A = w_1, w_2, w_3, \dots, w_n$ and $B = x_1, x_2, x_3, \dots, x_n$. Then there exists a non-empty set of integers $i_1, i_2, i_3, \dots, i_n$ such that, to solve the PCP we try all the combinations of $i_1, i_2, i_3, \dots, i_n$ to find the $w_i = x_i$ then we say that PCP has a soln.

Ques $A = (1, 0, 010, 11)$ $B = (10, 10, 01, 1)$
The input set is $\Sigma = \{0, 1\}$. Find the Solⁿ.

Solⁿ A Solⁿ is 1, 2, 1, 3, 3, 4 that means
 $w_1 w_2 w_1 w_3 w_3 w_4 = x_1 x_2 x_1 x_3 x_3 x_4$

The constructed string from both list is
10101001011

w_1	w_2	w_1	w_3	w_3	w_4
1	0	1	010	010	11
x_1	x_2	x_1	x_3	x_3	x_4
10	10	10	01	01	1

Read from L to R

Ques $A = \{ba, ab, a, baa, b\}$ $B = \{bab, baa, ba, a, aba\}$
The input set $\Sigma = \{a, b\}$

Solⁿ 1 5 2 3 4 4 3 4

Ques $A = \{100, 0, 1\}$ $B = \{1, 100, 00\}$

Solⁿ 1 3 1 1 3 2 2

Ques Does PCP with two lists $x = (b, bab^3, ba)$ and $y = (b^3, ba, a)$ have Solⁿ?

Ans 2 1 1 3 2 1 1 3
bab³ b b ba ba b³ b³ a

which form bab³b³a. Thus PCP has a Solⁿ.

Modified PCP:

If the first substring used in PCP is always x_1 and y_1 or (w_1, x_1) then the PCP is known as MPCP.

Given a set of pairs of strings $\{(w_1, x_1), (w_2, x_2), \dots, (w_n, x_n)\}$ is there then the solution is an instance such that $w_1 w_2 w_3 \dots w_n = x_1 x_2 x_3 \dots x_n$ that means this pair (w_1, x_1) is forced to be at the beginning of the strings. For e.g., consider two lists A and B as -

	List A	List B
i	w_i	x_i
1	11	111
2	100	001
3	111	11

$$\begin{bmatrix} 111 & 111 \\ 001 & 11 \\ 11 & 001 \end{bmatrix}$$

Then the solⁿ is

$$w_1 w_2 w_3 = x_1 x_2 x_3$$

$$11100111 = 11100111$$

That means it is essential to have w_1 and x_1 at the beginning of the list.

104